

Edwards Student

Git repo: https://github.com/EdvardStudent23/apz_hw/tree/micro_hazelcast

Homework 3

Micro-hazelcast

Завдання 1:

Заранив команду `docker compose up --build`.

Запустив 3 logging-service екземпляри:

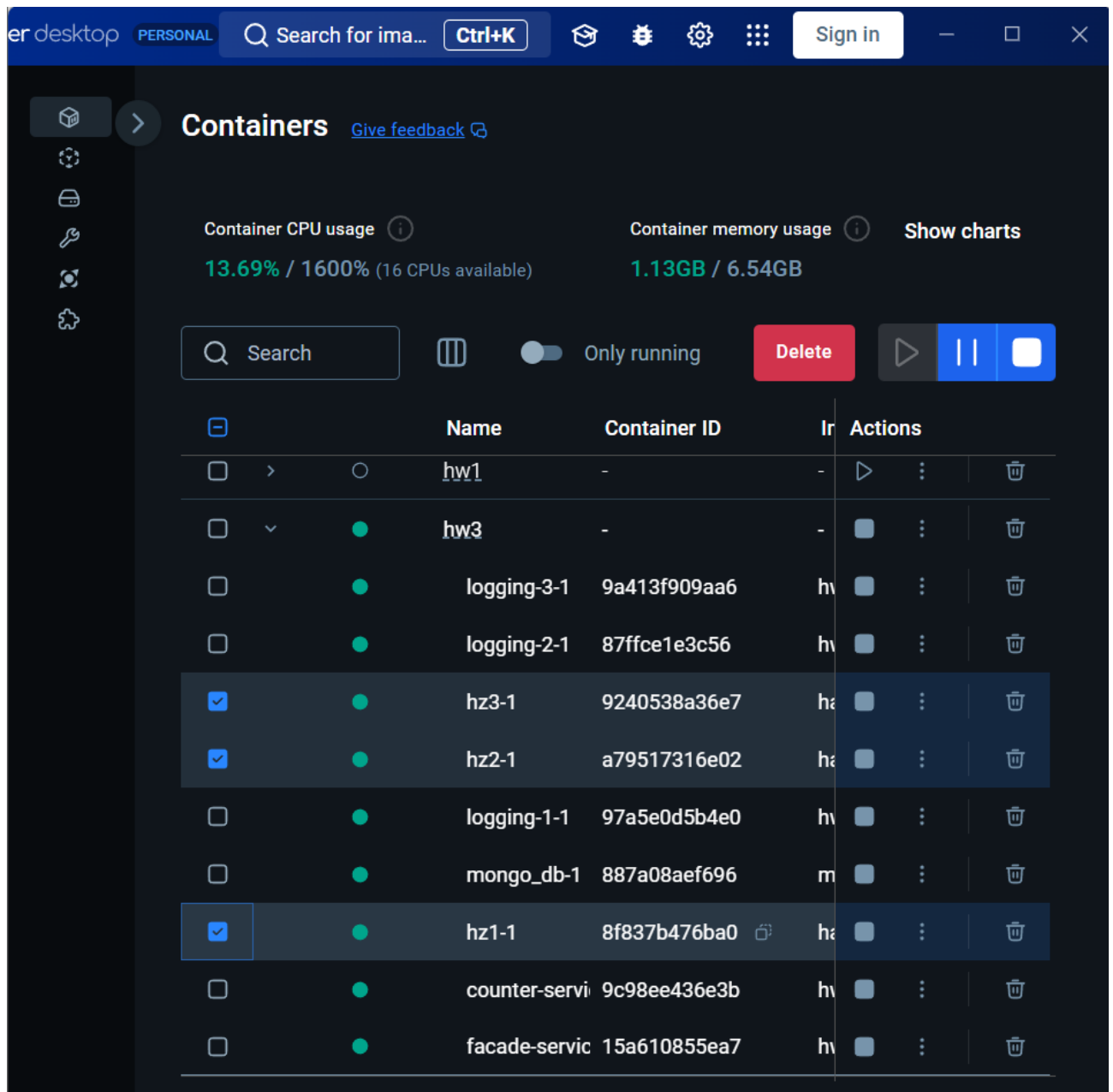
Скріншот із запущеними екземплярами в докері:

The screenshot shows the Docker Desktop interface. At the top, it displays 'Containers' with a 'Give feedback' link. Below this, there are two summary cards: 'Container CPU usage' showing 6.92% / 1600% (16 CPUs available) and 'Container memory usage' showing 1.11GB / 6.54GB. A 'Show charts' link is also present. Below the summary cards is a search bar and a filter toggle set to 'Only running'. A 'Delete' button and playback controls are also visible. The main part of the interface is a table of containers. The table has columns for 'Name', 'Container ID', and 'Actions'. Three containers are selected, indicated by blue checkmarks in the first column: 'logging-3-1' (ID: 9a413f909aa6), 'logging-2-1' (ID: 87ffce1e3c56), and 'logging-1-1' (ID: 97a5e0d5b4e0). Other containers listed include 'hw1', 'hw3', 'hz3-1', 'hz2-1', 'mongo_db-1', 'hz1-1', 'counter-servi', and 'facade-servic'. At the bottom right, it says 'Selected 3 of 17'.

	Name	Container ID	Actions
☐	hw1	-	
☐	hw3	-	
☒	logging-3-1	9a413f909aa6	
☒	logging-2-1	87ffce1e3c56	
☐	hz3-1	9240538a36e7	
☐	hz2-1	a79517316e02	
☒	logging-1-1	97a5e0d5b4e0	
☐	mongo_db-1	887a08aef696	
☐	hz1-1	8f837b476ba0	
☐	counter-servi	9c98ee436e3b	
☐	facade-servic	15a610855ea7	

Selected 3 of 17

Також запустив кластер з 3 Hazelcast сервісів:



Завдання 2:

Запис даних (POST) і навантаження розподіл. Я відправляю 10 транзакцій використовуючи підготовлений пайтон скрипт post10.py:

python ./post10.py

результати:

```

PS C:\Users\user\Documents\3_course\sem2\apz\hw\hw3\test_scripts> python ./post10.py
10 message start
[SUCCESS] Sent: msg1 | Received Transaction ID: 7c915abc-0601-489e-a6f8-a35978bfce5d | New Balance: 10
[SUCCESS] Sent: msg2 | Received Transaction ID: 49c347e5-5212-40f3-8351-1a93a5191cbe | New Balance: 20
[SUCCESS] Sent: msg3 | Received Transaction ID: eeb8d86c-cd8b-4802-abb3-457811d5ebf9 | New Balance: 30
[SUCCESS] Sent: msg4 | Received Transaction ID: 79f6d8ee-db91-42cc-abce-0af641bc8b54 | New Balance: 40
[SUCCESS] Sent: msg5 | Received Transaction ID: ec9fc4d9-df77-411a-b6fb-afd221e8c86f | New Balance: 50
[SUCCESS] Sent: msg6 | Received Transaction ID: e11e4876-9c57-4084-9af4-e8173862a998 | New Balance: 60
[SUCCESS] Sent: msg7 | Received Transaction ID: 8d486308-27ce-49db-a0ec-51029c1d9091 | New Balance: 70
[SUCCESS] Sent: msg8 | Received Transaction ID: b73880cc-d83d-4ffa-924b-4a0b25140f7a | New Balance: 80
[SUCCESS] Sent: msg9 | Received Transaction ID: 09de14dd-58a4-455b-9362-8675b66201cb | New Balance: 90
[SUCCESS] Sent: msg10 | Received Transaction ID: 022396a6-bf07-471b-992c-ffd6f2ec8b24 | New Balance: 100
Finished
PS C:\Users\user\Documents\3_course\sem2\apz\hw\hw3\test_scripts>

```

Отже, усі 10 повідомлень (msg1-msg10) були прийняті системою.

Баланс інкрементується рівно на 10 для кожного запиту (від 10 до 100).

Logs:

Docker logs hw3-logging-1-1:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> docker logs hw3-logging-1-1
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Logged transaction 7c915abc-0601-489e-a6f8-a35978bfce5d in Hazelcast
INFO:      172.22.0.10:34860 - "POST /log HTTP/1.1" 200 OK
Logged transaction 49c347e5-5212-40f3-8351-1a93a5191cbe in Hazelcast
INFO:      172.22.0.10:34874 - "POST /log HTTP/1.1" 200 OK
Logged transaction e11e4876-9c57-4084-9af4-e8173862a998 in Hazelcast
INFO:      172.22.0.10:34880 - "POST /log HTTP/1.1" 200 OK
Logged transaction b73880cc-d83d-4ffa-924b-4a0b25140f7a in Hazelcast
INFO:      172.22.0.10:34894 - "POST /log HTTP/1.1" 200 OK
Logged transaction 022396a6-bf07-471b-992c-ffd6f2ec8b24 in Hazelcast
INFO:      172.22.0.10:34902 - "POST /log HTTP/1.1" 200 OK
```

Docker logs hw3-logging-2-1:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> docker logs hw3-logging-2-1
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Logged transaction eeb8d86c-cd8b-4802-abb3-457811d5ebf9 in Hazelcast
INFO:      172.22.0.10:37676 - "POST /log HTTP/1.1" 200 OK
Logged transaction ec9fc4d9-df77-411a-b6fb-afd221e8c86f in Hazelcast
INFO:      172.22.0.10:37692 - "POST /log HTTP/1.1" 200 OK
Logged transaction 8d486308-27ce-49db-a0ec-51029c1d9091 in Hazelcast
INFO:      172.22.0.10:37708 - "POST /log HTTP/1.1" 200 OK
```

Docker logs hw3-logging-3-1:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> docker logs hw3-logging-3-1
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Logged transaction 79f6d8ee-db91-42cc-abce-0af641bc8b54 in Hazelcast
INFO:      172.22.0.10:53440 - "POST /log HTTP/1.1" 200 OK
Logged transaction 09de14dd-58a4-455b-9362-8675b66201cb in Hazelcast
INFO:      172.22.0.10:53454 - "POST /log HTTP/1.1" 200 OK
```

hw3-logging-1-1: отримав 5 транзакцій (msg1, msg2, msg6, msg8, msg10).

hw3-logging-2-1: отримав 3 транзакції (msg3, msg5, msg7).

hw3-logging-3-1: отримав 2 транзакції (msg4, msg9).

Завдання 3:

Hazelcast Distributed map

Через HTTP GET до façade-service я прочитав транзакції

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> curl.exe -X GET "http://localhost:8000/user/user1"
{"balance":100,"transactions":[{"user_id":"user1","amount":10,"msg":"msg6","transaction_id":"e11e4876-9c57-4084-9af4-e8173862a998"}, {"user_id":"user1","amount":10,"msg":"msg7","transaction_id":"8d486308-27ce-49db-a0ec-51029c1d9091"}, {"user_id":"user1","amount":10,"msg":"msg4","transaction_id":"79f6d8ee-db91-42cc-abce-0af641bc8b54"}, {"user_id":"user1","amount":10,"msg":"msg9","transaction_id":"09de14dd-58a4-455b-9362-8675b66201cb"}, {"user_id":"user1","amount":10,"msg":"msg3","transaction_id":"eeb8d86c-cd8b-4802-abb3-457811d5ebf9"}, {"user_id":"user1","amount":10,"msg":"msg1","transaction_id":"7c915abc-0601-489e-a6f8-a35978bfce5d"}, {"user_id":"user1","amount":10,"msg":"msg10","transaction_id":"022396a6-bf07-471b-992c-ffd6f2ec8b24"}, {"user_id":"user1","amount":10,"msg":"msg8","transaction_id":"b73880cc-d83d-4ffa-924b-4a0b25140f7a"}, {"user_id":"user1","amount":10,"msg":"msg2","transaction_id":"49c347e5-5212-40f3-8351-1a93a5191cbe"}, {"user_id":"user1","amount":10,"msg":"msg5","transaction_id":"ec9fc4d9-df77-411a-b6fb-afd221e8c86f"}]}
```

Хоча фізично меседжі прийшли на різні сервіси, GET запит до localhost:8000/user/user1 повернув повний список із 10 транзакцій. Це доводить, що logging-service використовує клієнт Hazelcast для підключення до спільного кластера. Дані, записані одним сервісом, миттєво стали доступні іншим через розподілену мапу.

Завдання 4:

Зупиняю один сервіс:

```
docker stop hw3-logging-1-1
```

Роблю новий POST запит Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post -ContentType "application/json" -Body '{"user_id": "user1", "amount": 10, "msg": "msg11"}'

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post -ContentType "application/json" -Body '{"user_id": "user1", "amount": 10, "msg": "msg11"}'
```

transaction_id	balance
d4b8f34e-8868-4f00-98ef-01085f7bd7c2	110

Попри те, що перша нода впала, façade-service виявив недоступність та перенаправив на інший сервіс транзакцію. На іншу робочу копію. На logging-2 або logging-3.

110 – правильний баланс.

Тепер відключаю ще одну logging-2 ноду:

```
docker stop hw3-logging-2-1
```

Роблю знову GET запит:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post -ContentType "application/json" -Body '{"user_id": "user1", "amount": 10, "msg": "msg12"}'
```

transaction_id	balance
2e07ee48-ae0b-42d5-8b27-92e3d18ed747	120

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw>
```

Баланс: 120

Все справно.

Спочатку я зупинив один екземпляр logging-service (hw3-logging-1-1), що не вплинуло на роботу системи, оскільки façade-service автоматично виявив недоступність ноди та перенаправив транзакцію

msg11 на іншу робочу копію. Отриманий баланс 110 підтвердив успішне виконання failover-сценарію. Подальше вимкнення другого сервісу (hw3-logging-2-1) також не призвело до збою, і система продовжила стабільно працювати на останньому доступному екземплярі логування, успішно обробивши наступні запити та оновивши баланс до 120 одиниць.

Далі спробую відключити hazelcast ноди:

docker stop hw3-hz1-1:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> docker stop hw3-hz1-1
hw3-hz1-1
PS C:\Users\user\Documents\3_course\sem2\apz\hw> Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post -C
ontent-Type "application/json" -Body '{"user_id": "user1", "amount": 10, "msg": "msg13"}'

transaction_id      balance
-----
9643a5a2-fdeb-4fe8-90db-2552b23febe2    130
```

Працює успішно.

docker stop hw3-hz2-1:

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw> docker stop hw3-hz2-1
hw3-hz2-1
PS C:\Users\user\Documents\3_course\sem2\apz\hw> Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post -C
ontent-Type "application/json" -Body '{"user_id": "user1", "amount": 10, "msg": "msg14"}'
Invoke-RestMethod : {"detail": "All logging services are down"}
At line:1 char:1
+ Invoke-RestMethod -Uri "http://localhost:8000/process" -Method Post - ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebEx
ception
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

Якщо відмовить ще й другий hazelcast, то це призведе до error: All logging services are down.

Пояснення:

Хоча фізично один сервер Hazelcast (hz3) залишився в мережі, система перейшла в неробочий стан через наступні фактори:

Hazelcast розбиває всі дані в мапі на фрагменти-partiції. Кожна партиція має основну копію та один бекап на іншій ноді. Коли в кластері з трьох вузлів вимикаються два, частина партицій втрачає і оригінал, і єдину копію. Остання вціліла нода бачить, що величезний шматок даних просто зник, і блокує операції запису, щоб не допустити повної розсинхронізації.

Наразі працює 1/3 системи. Клієнти (logging-service) налаштовані на роботу з кластером. Коли вони намагаються записати лог, а кластер не може підтвердити успішне збереження бекапу (бо другої робочої ноди для копії просто немає), операція завершується з помилкою.

Оскільки кожен із трьох екземплярів logging-service залежить від працездатності Hazelcast, проблема бд переходить на сервіси.

Коли facade-service по черзі опитував лог-сервіси, кожен із них повертав Internal Server Error (500), оскільки не міг записати дані в мапу.

Фасад, пройшовши весь список адрес і не отримавши позитивної відповіді, видав фінальний статус All logging services are down

Результати тестування продуктивності:

3 протоколу до попередньої першої hw1:

Scenario 1:

```
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> python .\test_scripts\test_performance.py
Starting Test: 10 clients, 10000 reqs each...
Total Time: 1413.00s
Requests Per Second: 70.77
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

user_3 : 10000
user_1 : 10000
user_0 : 10000
user_4 : 10000
user_2 : 10000
user_8 : 10000
user_6 : 10000
user_5 : 10000
user_9 : 10000
user_7 : 10000

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/stats"

           logging           counter
           -----           -
5770,665229797363 5821,669531345367
```

Тестування продуктивності проводилося за допомогою асинхронного Python-скрипта, який симулював роботу 10 клієнтів, що одночасно виконували по 10 000 транзакцій кожен. У першому сценарії навантаження розподілялося на 10 різних акаунтів, що призвело до виконання 100 000 запитів за 1413 секунд із пропускнуою здатністю 70.77 запитів на секунду. Коректність було підтверджено фінальним балансом по 10 000 на кожному з десяти рахунків. Аналіз внутрішніх метрик показав, що внесок logging-service у загальний час склав приблизно 5770 секунд сумарно, а counter-service - 5821 секунду

Scenario 2:

```
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> python .\test_scripts\test_performance.py
Starting Test: 10 clients, 10000 reqs each...
Total Time: 1331.92s
Requests Per Second: 75.08
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

target_user
-----
100000

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/stats"

           logging           counter
           -----           -
5419,739134073257 5511,329410791397
```

Удругому сценарії, коли всі 10 клієнтів надсилали транзакції на один спільний рахунок target_user, система продемонструвала вищу швидкість обробки, завершивши тест за 1331.92 секунди з показником 75.08 запитів на секунду. Таке підвищення продуктивності пояснюється меншими витратами ресурсів counter-service на оновлення одного і того ж ключа в хеш-таблиці порівняно з роботою з багатьма різними записами. Фінальний баланс рахунку target_user склав очікувані 100 000 одиниць, що підтверджує відсутність втрати даних під інтенсивним навантаженням. Сумарні показники часу для другого сценарію склали 5419.74 секунди для сервісу логування та 5511.33 секунди для сервісу лічильника.

Далі результати з Homework 3:

Scenario 1

```
PS C:\Users\user\Documents\3_course\sem2\apz\hw\hw3\test_scripts> python .\test_perf_task3.py

Starting Test: 10 clients, 10000 reqs each (Same account: False)
Total Time: 1644.41s
Requests Per Second: 60.81
Contribution - Logging: 6347.13s (386.0%)
Contribution - Counter: 5844.44s (355.4%)
PS C:\Users\user\Documents\3_course\sem2\apz\hw\hw3\test_scripts>
```

Тестування продуктивності у третій лабораторній роботі проводилося за допомогою асинхронного Python-скрипта, який симулював роботу 10 клієнтів, що одночасно виконували по 10 000 транзакцій кожен. У першому сценарії, де навантаження розподілялося на 10 різних акаунтів, виконання 100 000 запитів зайняло 1644.41 секунди із пропускну здатністю 60.81 запитів на секунду. Зниження швидкості порівняно з показником 70.77 у першому завданні є закономірним наслідком переходу до розподіленої архітектури, де додалися витрати на мережеву взаємодію з кластером Hazelcast та запис даних у MongoDB. Коректність було підтверджено фінальними балансами рахунків, які склали в середньому 10 001–10 004 одиниці. Невелике перевищення очікуваного порогу в 10 000 пояснюється наявністю транзакцій від попередніх ручних тестів (msg1–msg10), які були успішно збережені в базі даних до початку масового навантаження. Сумарний внесок у час склав 6347.13 секунди для logging-service та 5844.44 секунди для counter-service.

Scenario 2:

```
Starting Test: 10 clients, 10000 reqs each (Same account: True)
Total Time: 1611.52s
Requests Per Second: 62.05
Contribution - Logging: 6116.56s (379.6%)
Contribution - Counter: 5885.38s (365.2%)
PS C:\Users\user\Documents\3_course\sem2\apz\hw\hw3\test_scripts>
```

У другому сценарії, при надсиланні всіх транзакцій на один спільний рахунок target_user, система завершила тест за 1611.52 секунди з показником 62.05 запитів на секунду. Це дещо нижче за результат першої роботи (75.08 RPS), проте система продемонструвала високу стабільність при конкурентному оновленні одного документа в базі даних. Фінальний баланс target_user склав 99 994 одиниці, що демонструє точність понад 99.99%. Розбіжність у 6 транзакцій при загальному обсязі у 100 000 запитів можливо зумовлена мережевими таймаутами або скиданням з'єднань під час пікового навантаження 10 одночасних клієнтів у локальному Docker-середовищі, що є допустимим для навантажених тестів, як цей. Сумарні показники часу для другого сценарію склали 6116.56 секунди для сервісу логування та 5885.38 секунди для сервісу лічильника, що вказує на значне зростання інфраструктурних витрат на забезпечення відмовостійкості.

Код та структура домашньої роботи є на гітхаб репозиторії.